Project#2: Reinforcement Learning

1. Project Overview

- In this project, you will implement **Value Iteration** and **Q-learning**. You will test your agents first on Gridworld, then apply them to a simulated robot controller (Crawler) and Pacman.
- You will get **six** different tasks within this project
- This assignment reuses **UC Berkeley's Pacman AI project** (https://ai.berkeley.edu/), but with minimum adjustments for CAU's CSE 17182.

2. Environment

- Projects in this class require Python 3.6+.
- For the grading, I will use vanilla Python 3.9 in a `conda` environment. If you want the same setup, install `conda` and:

```
conda create -n pacman python=3.9 -y
conda activate pacman
python --version   # should show 3.9.x
```

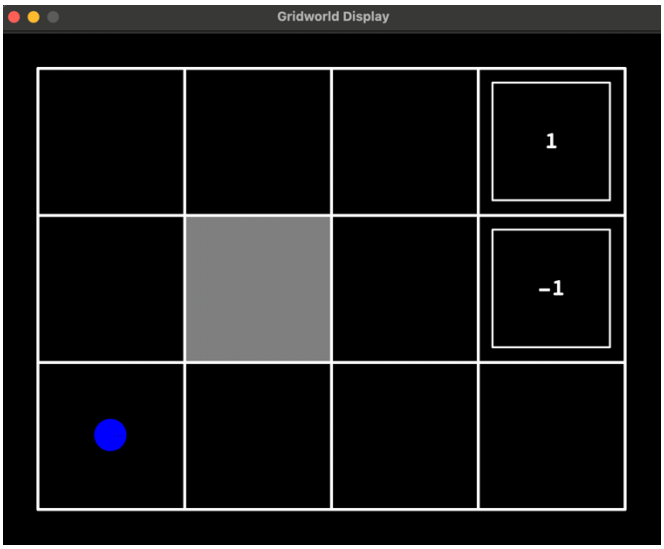
3. **Submission Deliverable & Deadline**

- Deadline: Dec 12 (Fri) 23:59 (through the e-class system)
- Within the code files from the given zip archive (`project2.zip`), you are allowed to edit ONLY the following three files (please do not change the filenames).
  - `valueIterationAgents.py`
  - `qlearningAgents.py`
  - `analysis.py`
- As a submission, you need to make the three files compressed and submit as ONE zip file with a filename `submission.zip`.
- If you violate this deliverable rule, huge penalty will be applied to your score.

4. Auto-grading

- You can try to grade your answers on your machine. This can be done:
  - `python autograder.py` (Run on all questions)
  - `python autograder.py -q q2` (Run on a particular question)

5. Starter: Try the Gridworld MDP



- The given files provide you to run Gridworld in manual control mode
  - `python gridworld.py -m` (with using the arrow keys)

- In this project, you will build your own agents (with Value Iteration or Q-learning); after the implementation you can test your agents as follows (the following code will not work until you implement those agents):
  - `python gridworld.py -a value` (Value Iteration; Q1)
  - `python gridworld.py -a q` (Q-learning; Q3)

- You can control many aspects of the simulation. A full list of options is available by running:
  - `python gridworld.py -h`

6. Other information

- Within the zip archive, you may want to read the following five files of code:
  - `mdp.py`: Defines methods on general MDPs.
  - `learningAgnets.py`: Defines the base classes `ValueEstimationAgent` and `QLearningAgent`, which your agents will extend.
  - `util.py`: utilities, including `util.Counter`, which is particularly useful for Q-learners.
  - `gridworld.py`: The Gridworld implementation.
  - `featureExtractors.py`: Classes for extracting features on (state, action) pairs. Used for the approximate Q-learning agent (in `qlearningAgents.py`).
- You can neglect other supporting files (e.g., `environment.py`)

Q1 (5 pts). Value Iteration

Update this file: `valueIterationAgents.py`

Recall the value iteration state update equation:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s')\right]$$

Write a value iteration agent in `ValueIterationAgent`, which has been partially specified for you in `valueIterationAgents.py`. `ValueIterationAgent` takes an MDP on construction and runs value iteration for the specified number of iterations (= argument `iterations`) before the constructor returns.

Value iteration computes k-step estimates of the optimal values, V_k. In addition to `runValueIteration`, implement the following methods for `ValueIterationAgent` using V_k.

- `computeActionFromValues(state)` computes the best action according to the value function given by `self.values`.
- `computeQValueFromValues(state, action)` returns the Q-value of the (state, action) pair given by the value function given by `self.values`.

Note: A policy synthesized from values of depth k (which reflect the next k rewards) will actually reflect the next $k+1$ rewards (i.e. you return π_{k+1}). Similarly, the Q-values will also reflect one more reward than the values (i.e. you return Q_{k+1}).

As a computed action, you should return the synthesized policy π_{k+1}.

Hint: You may optionally use the `util.Counter` class in `util.py`, Counter which is a dictionary with a default value of zero. However, be careful with `argmax`: the actual argmax you want may be a key not in the counter!

Note: Make sure to handle the case when a state has no available actions in an MDP.

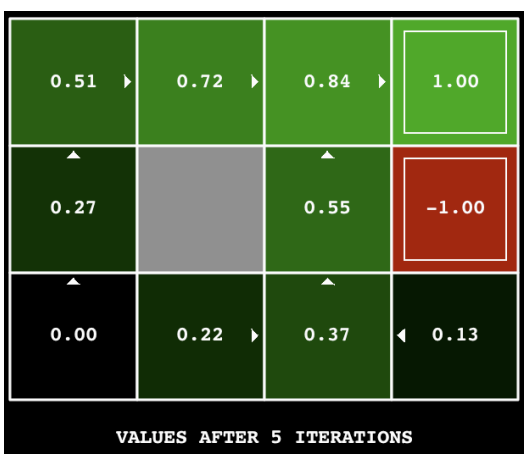
Test your implementation:

The following command loads your `ValueIterationAgent`, which will compute a policy for 100 iteration updates (option `-i`) and execute it 10 times (option `-k`).

- `python gridworld.py -a value -i 100 -k 10`

On the default grid, running value iteration for 5 iterations should give you this output.

- `python gridworld.py -a value -i 5`



Grading

- Your value iteration agent will be graded on a new grid. We will check your values, Q-values, and policies after fixed numbers of iterations and at convergence (e.g. after 100 iterations).
- You can run the autograder to check your points

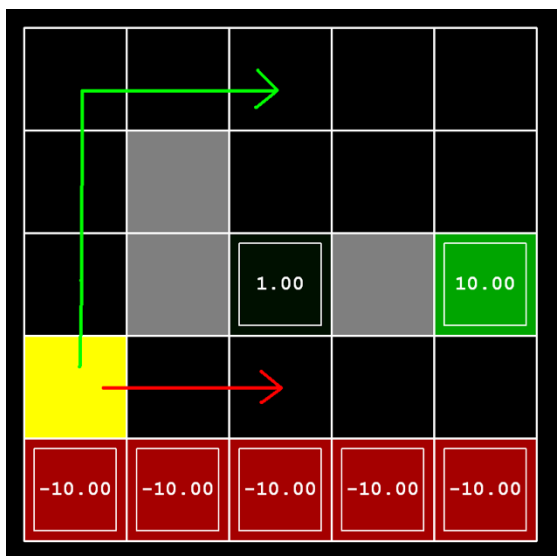
```
python autograder.py -q q1
```

Q2 (5 pts). Policies

Update this file: `analysis.py`

Consider the `DiscountGrid` layout, shown below. This grid has two terminal states with positive payoff (in the middle row), a close exit with payoff +1 and a distant exit with payoff +10. The bottom row of the grid consists of terminal states with negative payoff (shown in red); each state in this "cliff" region has payoff -10. The starting state is the yellow square.

We distinguish between two types of paths: (1) paths that "risk the cliff" and travel near the bottom row of the grid; these paths are shorter but risk earning a large negative payoff, and are represented by the red arrow in the figure below. (2) paths that "avoid the cliff" and travel along the top edge of the grid. These paths are longer but are less likely to incur huge negative payoffs. These paths are represented by the green arrow in the figure below.



In this question, you will choose settings of the **discount, noise, and living reward parameters** for this MDP to produce optimal policies of several different types.

Your setting of the parameter values for each part should have the property that, if your agent followed its optimal policy without being subject to any noise, it would exhibit the given behavior. If a particular behavior is not achieved for any setting of the parameters, assert that the policy is impossible by returning the string 'NOT POSSIBLE'.

Here are the optimal policy types you should attempt to produce:

2a) Prefer the close exit (+1), risking the cliff (-10)

2b) Prefer the close exit (+1), but avoiding the cliff (-10)

2c) Prefer the distant exit (+10), risking the cliff (-10)

2d) Prefer the distant exit (+10), avoiding the cliff (-10)

2e) Avoid both exits and the cliff (so an episode should never terminate)

To see what behavior a set of numbers ends up in, run the following command:

- `python gridworld.py -g DiscountGrid -a value --discount [YOUR_DISCOUNT] --noise [YOUR_NOISE] --livingReward [YOUR_LIVING_REWARD]`

Note: `question2a()` through `question2e()` should each return a 3-item tuple of `(discount, noise, living reward)` in `analysis.py`.

Note: You can check your policies in the GUI. For example, using a correct answer to 2a: the arrow in (0,1) should point east, the arrow in (1,1) should also point east, and the arrow in (2,1) should point north.

Note: On some machines you may not see an arrow. In this case, press a button on the keyboard to switch to qValue display, and mentally calculate the policy by taking the arg max of the available qValues for each state.

Grading

- We will check that the desired policy is returned in each case.
- You can run the autograder to check your points

```
python autograder.py -q q2
```

Q3 (5 pts). Q-Learning

Update this file: `qlearningAgents.py`

You will now write a Q-learning agent, which learns by trial and error from interactions with the environment through its `update(state, action, nextState, reward)` method. A stub of a Q-learner is specified in `QLearningAgent` in `qlearningAgents.py`.

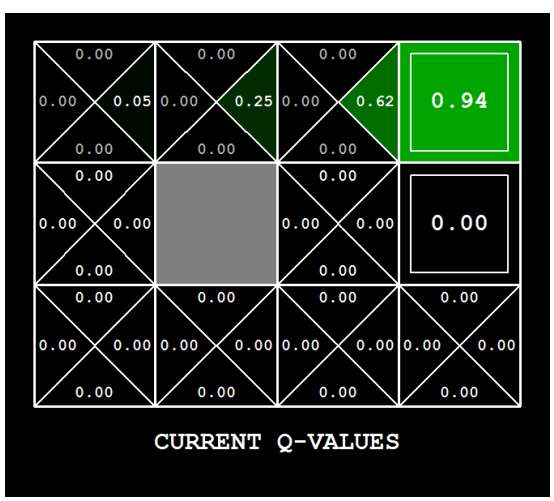
For this question, you must implement the `update`, `computeValueFromQValues`, `getQValue`, and `computeActionFromQValues` methods.

Note: For `computeActionFromQValues`, you should break ties randomly for better behavior. The `random.choice()` function will help. In a particular state, actions that your agent hasn't seen before still have a Q-value, specifically a Q-value of zero, and if all of the actions that your agent has seen before have a negative Q-value, an unseen action may be optimal.

Important: Make sure that in your `computeValueFromQValues` and `computeActionFromQValues` functions, you only access Q values by calling `getQValue`. This abstraction will be useful for later Q6 when you override `getQValue` to use features of state-action pairs rather than state-action pairs directly.

Hint: With the Q-learning update in place, you can watch your Q-learner learn under manual control. To help with debugging, you can turn off noise by using the `--noise 0.0` parameter. If you manually steer Pacman north and then east along the optimal path for four episodes (-k 4), you should see the following Q-values:

- `python gridworld.py -a q -k 4 --noise 0.0 -m`



Grading

- We will run your Q-learning agent and check that it learns the same Q-values and policy as our reference implementation when each is presented with the same set of examples.
- To grade your implementation, run the autograder

```
python autograder.py -q q3
```

Q4 (2 pts). Epsilon Greedy

Update this file: `qlearningAgents.py`

Complete your Q-learning agent by implementing epsilon-greedy action selection in `getAction`, meaning it chooses random actions an epsilon fraction of the time, and follows its current best Q-values otherwise. Note that choosing a random action may result in choosing the best action - that is, you should not choose a random sub-optimal action, but rather any random legal action.

You can choose an element from a list uniformly at random by calling the `random.choice` function. You can simulate a binary variable with probability `p` of success by using `util.flipCoin(p)`, which returns `True` with probability `p` and `False` with probability `1-p`.

After implementing the `getAction` method, observe the following behavior of the agent in Gridworld (with epsilon=0.3).

- `python gridworld.py -a q -k 100`

Your final Q-values should resemble those of your value iteration agent, especially along well-traveled paths. However, your average returns will be lower than the Q-values predict because of the random actions and the initial learning phase.

You can also observe the following simulations for different epsilon values. Does that behavior of the agent match what you expect?

- `python gridworld.py -a q -k 100 --noise 0.0 -e 0.1`
- `python gridworld.py -a q -k 100 --noise 0.0 -e 0.9`

With no additional code, you should now be able to run a Q-learning crawler robot:

- `python crawler.py`

If this doesn't work, you've probably written some code too specific to the GridWorld problem and you should make it more general to all MDPs.

This will invoke the crawling robot from class using your Q-learner. Play around with the various learning parameters to see how they affect the agent's policies and actions. Note that the step delay is a parameter of the simulation, whereas the learning rate and epsilon are parameters of your learning algorithm, and the discount factor is a property of the environment.

Grading

- To grade your implementation, run the autograder

```
python autograder.py -q q4
```

Q5 (1 pts). Q-Learning and Pacman

(Optionally) update this file: `qlearningAgents.py`

Time to play some Pacman! Pacman will play games in two phases. In the first phase, training, Pacman will begin to learn about the values of positions and actions. Because it takes a very long time to learn accurate Q-values even for tiny grids, Pacman's training games run in quiet mode by default, with no GUI (or console) display. Once Pacman's training is complete, he will enter testing mode. When testing, Pacman's `self.epsilon` and `self.alpha` will be set to 0.0, effectively stopping Q-learning and disabling exploration, in order to allow Pacman to exploit his learned policy. Test games are shown in the GUI by default.

Without any code changes you should be able to run Q-learning Pacman for very tiny grids as follows:

- `python pacman.py -p PacmanQAgent -x 2000 -n 2010 -l smallGrid`

Note: `PacmanQAgent` is already defined for you in terms of the `QLearningAgent` you've already written. `PacmanQAgent` is only different in that it has default learning parameters that are more effective for the Pacman problem (epsilon=0.05, alpha=0.2, gamma=0.8).

Hint: If your `QLearningAgent` works for `gridworld.py` and `crawler.py` but does not seem to be learning a good policy for Pacman on `smallGrid`, it may be because your `getAction` and/or `computeActionFromQValues` methods do not in some cases properly consider unseen actions. In particular, because unseen actions have by definition a Q-value of zero, if all of the actions that have been seen have negative Q-values, an unseen action may be optimal. Beware of the `argMax` function from `util.Counter`!

Note: If you want to experiment with learning parameters, you can use the option `-a`, for example `-a epsilon=0.1,alpha=0.3,gamma=0.7`. These values will then be accessible as `self.epsilon`, `self.gamma` and `self.alpha` inside the agent.

Note: While a total of 2010 games will be played, the first 2000 games will not be displayed because of the option `-x 2000`, which designates the first 2000 games for training (no output). Thus, you will only see Pacman play the last 10 of these games. The number of training games is also passed to your agent as the option `numTraining`. If you want to watch 10 training games to see what's going on, use the command:

- `python pacman.py -p PacmanQAgent -n 10 -l smallGrid -a numTraining=10`

Grading

- You will receive full credit for this question if the command above works without exceptions and your agent wins at least 80% of the time.
- The autograder will run 100 test games after the 2000 training games.
- To grade your implementation, run the autograder

```
python autograder.py -q q5
```

Q6 (3 pts). Approximate Q-Learning

Update this file: `qlearningAgents.py`

Implement an approximate Q-learning agent that learns weights for features of states, where many states might share the same features. Write your implementation in `ApproximateQAgent` class in `qlearningAgents.py,` which is a subclass of `PacmanQAgent`.

Note: Approximate Q-learning assumes the existence of a feature function $f(s, a)$ over state and action pairs, which yields a vector $[f_1(s, a), \ldots, f_i(s, a), \ldots, f_n(s, a)]$ of feature values. We provide feature functions for you in `featureExtractors.py.` Feature vectors are `util.Counter` (like a dictionary) objects containing the non-zero pairs of features and values; all omitted features have value zero. So, instead of an vector where the index in the vector defines which feature is which, we have the keys in the dictionary define the idenity of the feature.

The approximate Q-function takes the following form:

$$Q(s, a) = \sum_{i=1}^{n} f_i(s, a) w_i$$

where each weight w_i is associated with a particular feature $f_i(s, a)$. In your code, you should implement the weight vector as a dictionary mapping features (which the feature extractors will return) to weight values. You will update your weight vectors similarly to how you updated Q-values:

$$w_i \leftarrow w_i + \alpha \cdot \text{difference} \cdot f_i(s, a)$$

$$\text{difference} = \left(r + \gamma \max_{a'} Q(s', a')\right) - Q(s, a)$$

Note that the difference term is the same as in normal Q-learning, and r is the experienced reward.

By default, `ApproximateQAgent` uses the `IdentityExtractor`, which assigns a single feature to every `(state,action)` pair. With this feature extractor, your approximate Q-learning agent should work identically to `PacmanQAgent`. You can test this with the following command:

- `python pacman.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid`

Important: `ApproximateQAgent` is a subclass of `QLearningAgent`, and it therefore shares several methods like `getAction`. Make sure that your methods in `QLearningAgent` call `getQValue` instead of accessing Q-values directly, so that when you override getQValue in your approximate agent, the new approximate q-values are used to compute actions.

Once you're confident that your approximate learner works correctly with the identity features, run your approximate Q-learning agent with our custom feature extractor, which can learn to win with ease:

- `python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumGrid`

If you have no errors, your approximate Q-learning agent should win almost every time with these simple features, even with only 50 training games.

Grading

- We will run your approximate Q-learning agent and check that it learns the same Q-values and feature weights as our reference implementation when each is presented with the same set of examples.
- To grade your implementation, run the autograder:

```
python autograder.py -q q6
```

If you find any error in the grammar or project description,
please contact the instructor via email: hsmoon@cau.ac.kr

Plagiarism in code will result in an automatic F grade, regardless of the reason. CAU has the internal system of detecting possible reuse of code. You may consult tools like ChatGPT for guidance, but do not copy or submit generated content as-is. This may lead to high similarity across submissions, which will be treated as plagiarism.